

Title : BANKERS ALGORITHM

Date : 13/02/26

ROLL NO: 24WH1A0584

Page No.: 22

Aim 3. Write a program to simulate Bankers Algorithm for Deadlock Avoidance and Prevention.

Program:

```
#include <stdio.h>
int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int total[m], avail[m];

    printf("\nEnter total instances of each resource:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &total[i]);
    }

    printf("\nEnter Max Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    for(j = 0; j < m; j++) {
        avail[j] = total[j];
        for(i = 0; i < n; i++) {
            avail[j] -= alloc[i][j];
        }
    }
    printf("\nAvailable Resources:\n");
```

BVRIT HYDERABAD College of Engineering for Women

Title : BANKERS ALGORITHM

Date :13/02/26

ROLL NO: 24WH1A0584

Page No.: 23

```
for(i = 0; i < m; i++) {
    printf("%d ", avail[i]);
}
printf("\n");

int finish[n], safeSeq[n];
for(i = 0; i < n; i++) {
    finish[i] = 0;
}

int count = 0;
while(count < n) {
    int found = 0;
    for(i = 0; i < n; i++) {
        if(finish[i] == 0) {
            int flag = 0;

            for(j = 0; j < m; j++) {
                if(need[i][j] > avail[j]) {
                    flag = 1;
                    break;
                }
            }

            if(flag == 0) {
                for(k = 0; k < m; k++) {
                    avail[k] += alloc[i][k];
                }

                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0) {
        printf("\nNo Safe Sequence\n");
        printf("System is NOT in Safe State (Deadlock)\n");
        return 0;
    }
}

printf("\nSafe Sequence: ");
for(i = 0; i < n; i++) {
    printf("P%d ", safeSeq[i]);
}

printf("\nSystem is in Safe State\n");

return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : BANKERS ALGORITHM

Date : 13/02/26

ROLL NO: 24WH1A0584

Page No.: 24

OUTPUT:

```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim bankers.c;
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc bankers.c;
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./a.out
Enter number of processes: 5
Enter number of resources: 3

Enter total instances of each resource:
10 5 7

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

Available Resources:
3 3 2

Safe Sequence: P1 P3 P4 P0 P2
System is in Safe State
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim bankers.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc bankers.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./a.out
Enter number of processes: 3
Enter number of resources: 3

Enter total instances of each resource:
1 1 1

Enter Max Matrix:
1 1 1
1 1 1
1 1 1

Enter Allocation Matrix:
1 0 0
0 1 0
0 0 1

Need Matrix:
0 1 1
1 0 1
1 1 0

Available Resources:
0 0 0

No Safe Sequence
System is NOT in Safe State (Deadlock)
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

Aim 4. Write a program to implement the Producer-Consumer problem using semaphores using UNIX/LINUX system calls.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int produce_count, consume_count;
sem_t empty, full, mutex;
int done = 0;

void *producer(void *arg) {
    int item;
    for(int i = 0; i < produce_count; i++) {
        item = rand() % 100;
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = item;
        printf("Produced: %d at %d\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);
    }
    done = 1;
    sem_post(&full);
    pthread_exit(NULL);
}

void *consumer(void *arg) {
    int item;
    int consumed = 0;
    while(consumed < consume_count) {
        sem_wait(&full);
        sem_wait(&mutex);
        if(done && in == out) {
            sem_post(&mutex);
            break;
        }
        item = buffer[out];
```

BVRIT HYDERABAD College of Engineering for Women

Title: PRODUCER-CONSUMER PROBLEM

Date : 20/02/2022

ROLL NO: 24WH1A0584

Page No.: 26

```
        printf("Consumed: %d from %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        consumed++;

        sem_post(&mutex);
        sem_post(&empty);

        sleep(1);
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t p, c;
    srand(time(0));

    printf("Enter number of items to produce: ");
    scanf("%d", &produce_count);

    printf("Enter number of items to consume: ");
    scanf("%d", &consume_count);

    if (produce_count > consume_count) {
        printf("Producer will produce more items than consumer will consume.\n");
    }
    else if (consume_count > produce_count) {
        printf("Consumer wants more items than producer will produce.\n");
    }
    else {
        printf("Producer and Consumer counts are equal.\n");
    }

    if (consume_count > produce_count) {
        printf("Adjusting consume count to %d to avoid waiting.\n",
produce_count);
        consume_count = produce_count;
    }

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    sem_destroy(&mutex);
    return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

Title: PRODUCER-CONSUMER PROBLEM

Date : 20/02/26

ROLL NO: 24WH1A0584

Page No.: 27

OUTPUT:

```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim producer_consumer.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc producer_consumer.c -o pc -pthread
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./pc
Enter number of items to produce: 3
Enter number of items to consume: 3
Producer and Consumer counts are equal.
Produced: 40 at 0
Consumed: 40 from 0
Produced: 87 at 1
Consumed: 87 from 1
Produced: 45 at 2
Consumed: 45 from 2
```

```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim producer_consumer.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc producer_consumer.c -o pc -pthread
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./pc
Enter number of items to produce: 1
Enter number of items to consume: 2
Consumer wants more items than producer will produce.
Adjusting consume count to 1 to avoid waiting.
Produced: 21 at 0
Consumed: 21 from 0
```

```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim producer_consumer.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc producer_consumer.c -o pc -pthread
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./pc
Enter number of items to produce: 2
Enter number of items to consume: 1
Producer will produce more items than consumer will consume.
Produced: 19 at 0
Consumed: 19 from 0
Produced: 50 at 1
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

Aim 5. Write a program to illustrate the following IPC mechanisms

a) Pipes b) FIFOs c) Message Queues d) Shared Memory

a) Pipes:

Program:

```
#include <stdio.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>

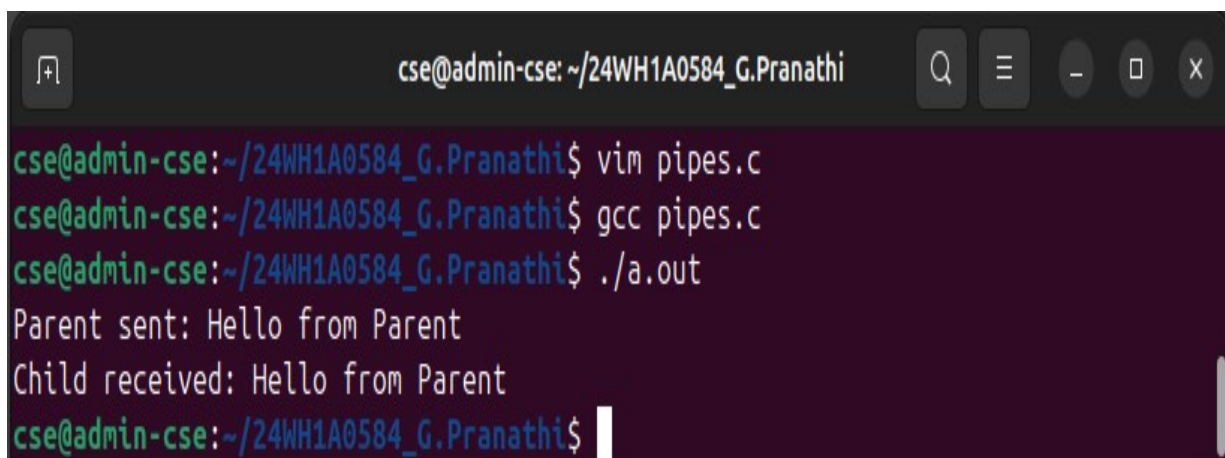
int main() {
    int fd[2];
    char msg[] = "Hello from Parent";
    char buffer[100];

    pipe(fd);

    if (fork() == 0) {
        // Child process reads
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
    } else {
        // Parent process writes
        close(fd[0]);
        write(fd[1], msg, strlen(msg) + 1);
        printf("Parent sent: %s\n", msg);
    }

    return 0;
}
```

OUTPUT:



```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim pipes.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc pipes.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./a.out
Parent sent: Hello from Parent
Child received: Hello from Parent
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

b) FIFOs:

fifo_writer.c Program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;
    char msg[] = "Message from FIFO Writer";
    fd = open("myfifo", O_WRONLY);
    write(fd, msg, strlen(msg) + 1);

    printf("Writer sent: %s\n", msg);
    close(fd);
    return 0;
}
```

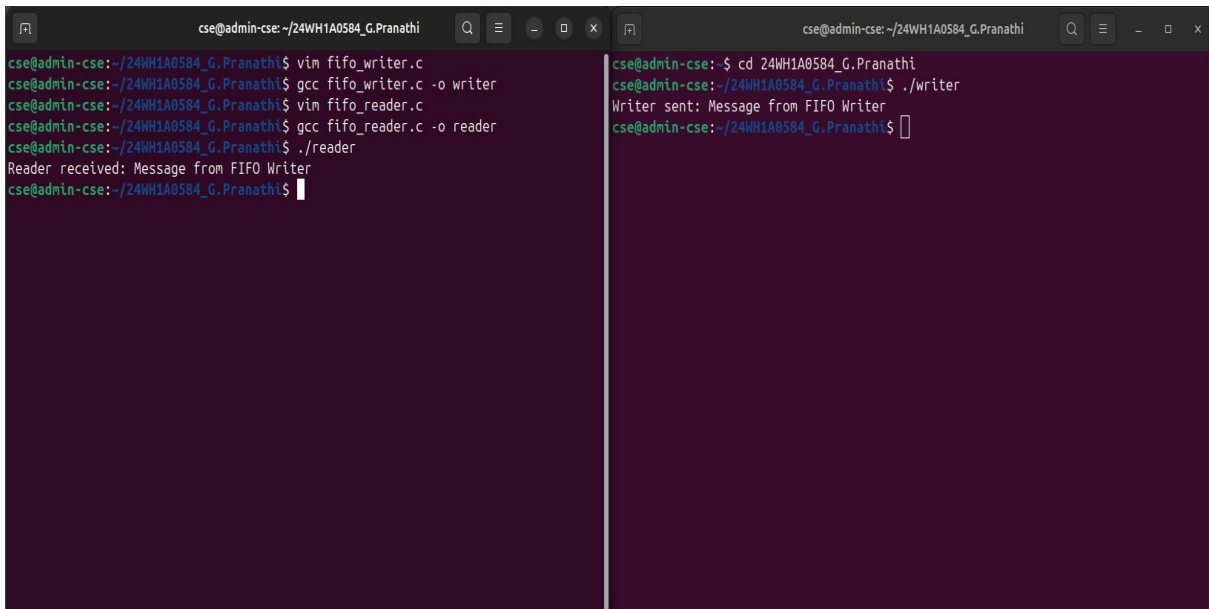
fifo_reader.c Program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd;
    char buffer[100];
    fd = open("myfifo", O_RDONLY);
    read(fd, buffer, sizeof(buffer));

    printf("Reader received: %s\n", buffer);
    close(fd);
    return 0;
}
```

OUTPUT:



The screenshot shows two terminal windows. The left window shows the compilation and execution of the FIFO programs. The right window shows the output of the writer program.

```
cse@admin-cse:~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim fifo_writer.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc fifo_writer.c -o writer
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim fifo_reader.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc fifo_reader.c -o reader
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./reader
Reader received: Message from FIFO Writer
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

```
cse@admin-cse:~/24WH1A0584_G.Pranathi$ cd 24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./writer
Writer sent: Message from FIFO Writer
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```


c) Message Queues:

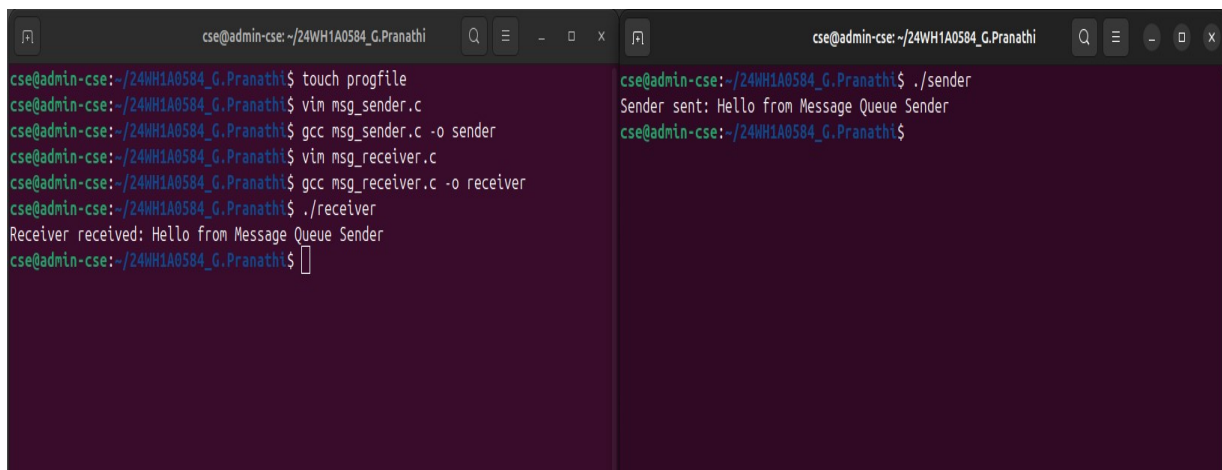
msg_sender.c Program:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
struct message {
    long msg_type;
    char msg_text[100];
};
int main() {
    key_t key = ftok("progfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct message msg;
    msg.msg_type = 1;
    strcpy(msg.msg_text, "Hello from Message Queue Sender");
    msgsnd(msgid, &msg, sizeof(msg.msg_text), 0);
    printf("Sender sent: %s\n", msg.msg_text);
    return 0;
}
```

msg_receiver.c Program:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
struct message {
    long msg_type;
    char msg_text[100];
};
int main() {
    key_t key = ftok("progfile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct message msg;
    msgrcv(msgid, &msg, sizeof(msg.msg_text), 1, 0);
    printf("Receiver received: %s\n", msg.msg_text);
    msgctl(msgid, IPC_RMID, NULL);
    return 0;
}
```

OUTPUT:



The screenshot shows two terminal windows side-by-side. The left window shows the compilation and execution of the sender and receiver programs. The right window shows the execution of the sender program, which outputs 'Sender sent: Hello from Message Queue Sender'.

```
cse@admin-cse:~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ touch progfile
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim msg_sender.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc msg_sender.c -o sender
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim msg_receiver.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc msg_receiver.c -o receiver
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./receiver
Receiver received: Hello from Message Queue Sender
cse@admin-cse:~/24WH1A0584_G.Pranathi$

cse@admin-cse:~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./sender
Sender sent: Hello from Message Queue Sender
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

d) Shared Memory :

shm_writer.c Program:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

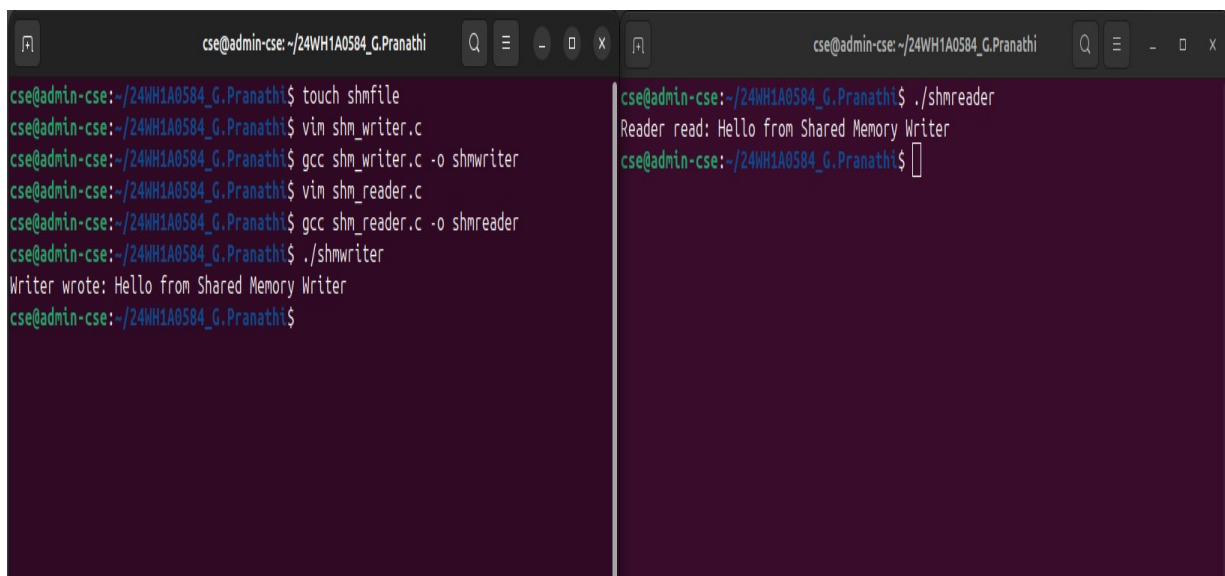
int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmatt(shmid, (void*)0, 0);
    strcpy(str, "Hello from Shared Memory Writer");
    printf("Writer wrote: %s\n", str);
    shmdt(str);
    return 0;
}
```

shm_reader.c Program:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>

int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char*) shmatt(shmid, (void*)0, 0);
    printf("Reader read: %s\n", str);
    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

OUTPUT :



```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ touch shmfile
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim shm_writer.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc shm_writer.c -o shmwriter
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim shm_reader.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc shm_reader.c -o shmreader
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./shmwriter
Writer wrote: Hello from Shared Memory Writer
cse@admin-cse:~/24WH1A0584_G.Pranathi$

cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./shmreader
Reader read: Hello from Shared Memory Writer
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

Aim 6. Write a program to simulate the following memory management techniques

a) Paging

b) Segmentation.

a) Paging:

Program:

```
#include <stdio.h>
int main() {
    int logical_memory, page_size;
    int num_pages;
    printf("Enter total logical memory (bytes): ");
    scanf("%d", &logical_memory);
    printf("Enter page size (bytes): ");
    scanf("%d", &page_size);
    num_pages = logical_memory / page_size;
    int page_table[num_pages];
    printf("\nNumber of pages = %d\n", num_pages);
    printf("\nEnter Page Table (Page -> Frame):\n");
    for (int i = 0; i < num_pages; i++) {
        printf("Page %d -> Frame: ", i);
        scanf("%d", &page_table[i]);
    }
    int page_number, offset;
    printf("\nEnter Page Number: ");
    scanf("%d", &page_number);
    printf("Enter Offset: ");
    scanf("%d", &offset);
    if (page_number >= num_pages || offset >= page_size) {
        printf("\nInvalid Page Number or Offset\n");
        return 0;
    }
    int frame_number = page_table[page_number];
    int physical_address = (frame_number * page_size) + offset;
    printf("Page Number = %d\n", page_number);
    printf("Offset = %d\n", offset);
    printf("Frame Number = %d\n", frame_number);
    printf("Physical Address = (%d * %d) + %d = %d\n",
        frame_number, page_size, offset, physical_address);
    return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

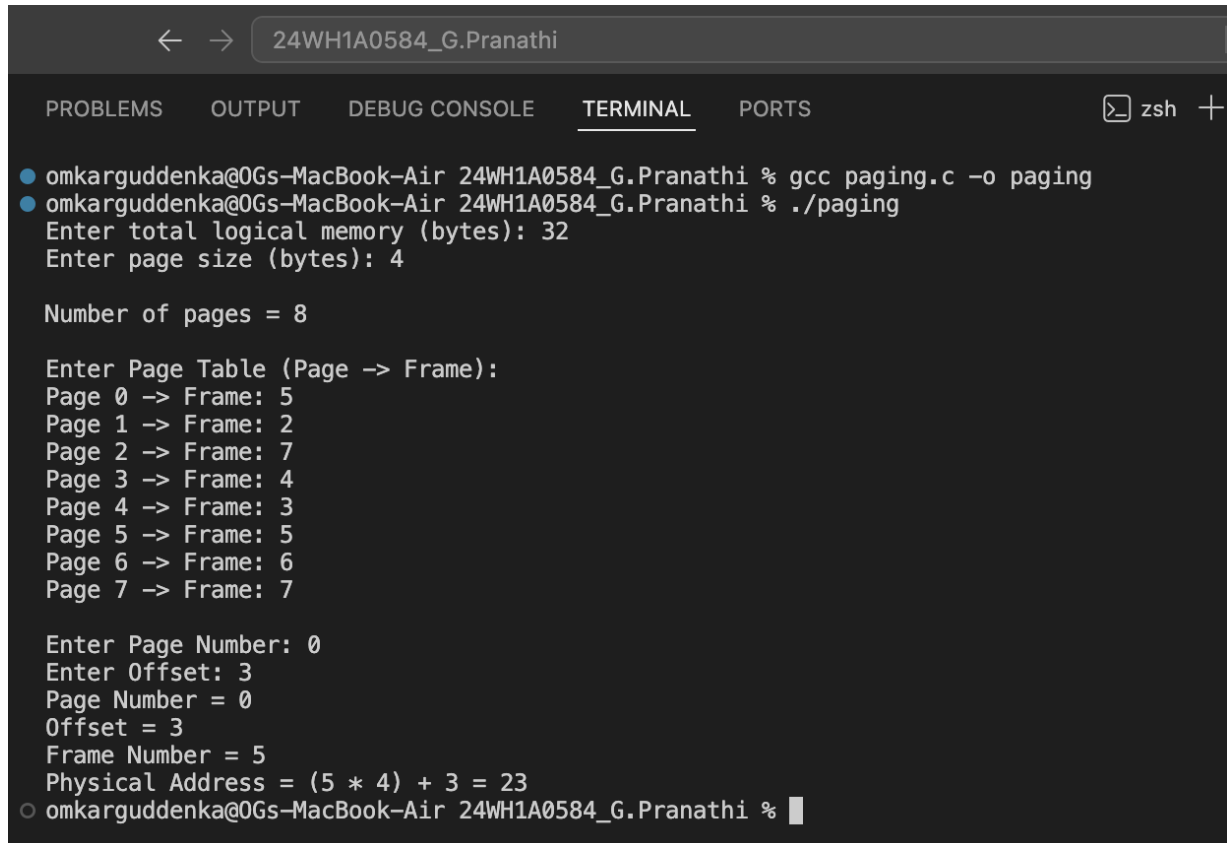
Title: MEMORY MANAGEMENT TECHNIQUES

Date : 20/03/26

ROLL NO: 24WH1A0584

Page No.: 33

OUTPUT :



```
← → 24WH1A0584_G.Pranathi
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh +
● omkarguddenka@0Gs-MacBook-Air 24WH1A0584_G.Pranathi % gcc paging.c -o paging
● omkarguddenka@0Gs-MacBook-Air 24WH1A0584_G.Pranathi % ./paging
Enter total logical memory (bytes): 32
Enter page size (bytes): 4

Number of pages = 8

Enter Page Table (Page -> Frame):
Page 0 -> Frame: 5
Page 1 -> Frame: 2
Page 2 -> Frame: 7
Page 3 -> Frame: 4
Page 4 -> Frame: 3
Page 5 -> Frame: 5
Page 6 -> Frame: 6
Page 7 -> Frame: 7

Enter Page Number: 0
Enter Offset: 3
Page Number = 0
Offset = 3
Frame Number = 5
Physical Address = (5 * 4) + 3 = 23
○ omkarguddenka@0Gs-MacBook-Air 24WH1A0584_G.Pranathi %
```

b) Segmentation:

```
#include <stdio.h>

int main() {
    int base[10], limit[10];
    int n, segNumber, offset;

    printf("Enter number of segments: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++) {
        printf("Segment %d Base: ", i);
        scanf("%d", &base[i]);

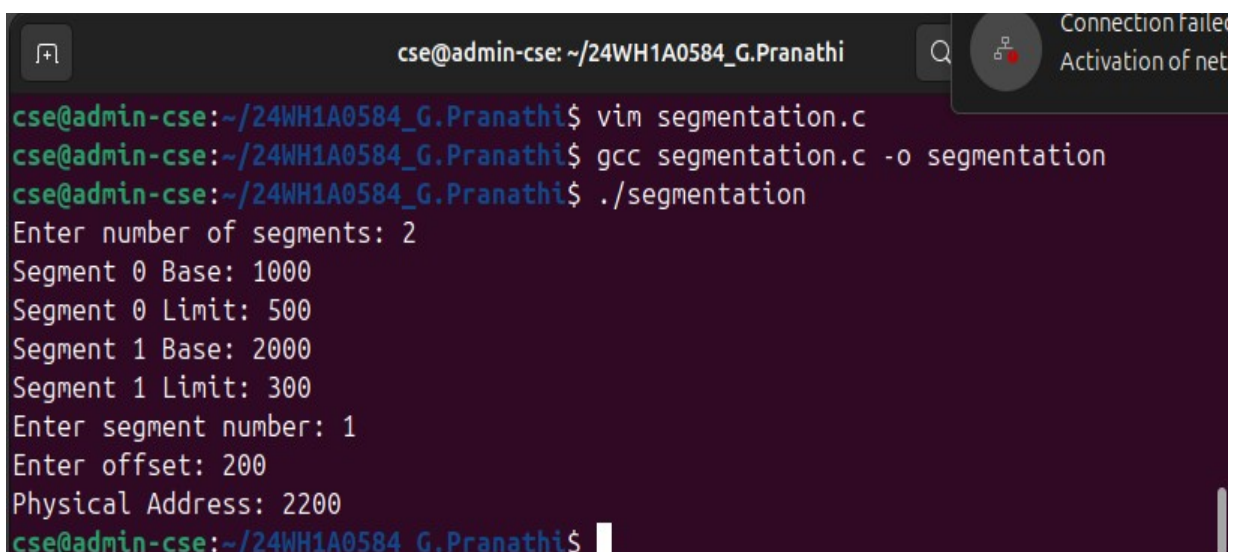
        printf("Segment %d Limit: ", i);
        scanf("%d", &limit[i]);
    }
    printf("Enter segment number: ");
    scanf("%d", &segNumber);

    printf("Enter offset: ");
    scanf("%d", &offset);

    if(segNumber >= n) {
        printf("Invalid segment number\n");
    } else if(offset >= limit[segNumber]) {
        printf("Segmentation Fault (offset exceeds limit)\n");
    } else {
        int physicalAddress = base[segNumber] + offset;

        printf("Physical Address: %d\n", physicalAddress);
    }
    return 0;
}
```

OUTPUT :



```
cse@admin-cse: ~/24WH1A0584_G.Pranathi
cse@admin-cse:~/24WH1A0584_G.Pranathi$ vim segmentation.c
cse@admin-cse:~/24WH1A0584_G.Pranathi$ gcc segmentation.c -o segmentation
cse@admin-cse:~/24WH1A0584_G.Pranathi$ ./segmentation
Enter number of segments: 2
Segment 0 Base: 1000
Segment 0 Limit: 500
Segment 1 Base: 2000
Segment 1 Limit: 300
Enter segment number: 1
Enter offset: 200
Physical Address: 2200
cse@admin-cse:~/24WH1A0584_G.Pranathi$
```

Aim 7. Write a program to simulate Contiguous Memory Allocation techniques

a) First-Fit

b) Best-Fit

c) Worst-fit

a) First-Fit :

Program:

```
#include <stdio.h>

int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];

    printf("Enter sizes of memory blocks:\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int process[n];

    printf("Enter sizes of processes:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &process[i]);

    int allocation[n];
    for(int i = 0; i < n; i++) allocation[i] = -1;

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) {
            if(blocks[j] >= process[i]) {
                allocation[i] = j;
                blocks[j] -= process[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");
    for(int i = 0; i < n; i++) {
        if(allocation[i] != -1)
            printf("Process %d -> Block %d\n", i+1, allocation[i]+1);
        else
            printf("Process %d -> Not Allocated\n", i+1);
    }

    return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

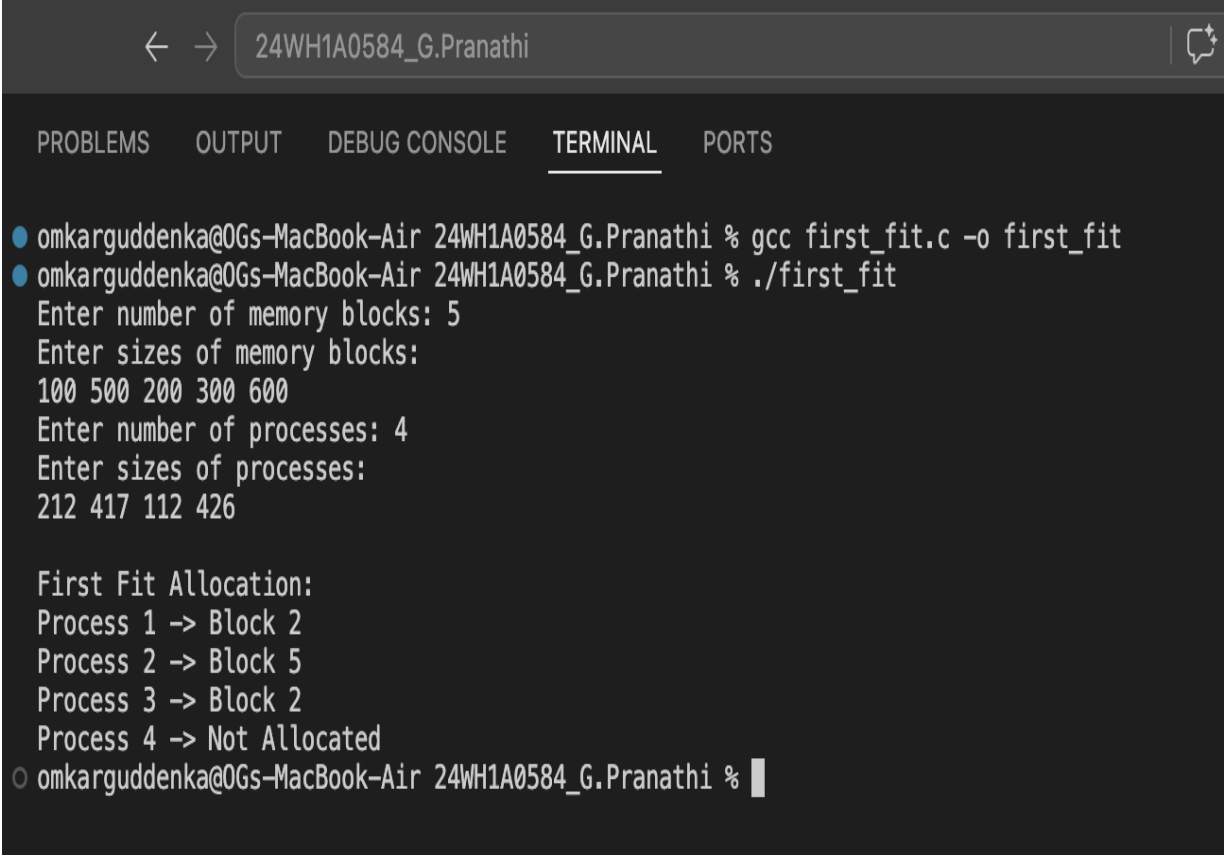
Title: CONTIGUOUS MEMORY ALLOCATION TECHNIQUES

Date : 10/04/26

ROLL NO: 24WH1A0584

Page No.: 36

OUTPUT:



```
← → 24WH1A0584_G.Pranathi

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc first_fit.c -o first_fit
• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./first_fit
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

First Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 5
Process 3 -> Block 2
Process 4 -> Not Allocated
• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

BVRIT HYDERABAD College of Engineering for Women

Title : CONTIGUOUS MEMORY ALLOCATION TECHNIQUES

Date : 10/04/26

ROLL NO: 24WH1A0584

Page No.: 37

b) Best-Fit:

Program:

```
#include <stdio.h>
int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];

    printf("Enter sizes of memory blocks:\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

    printf("Enter number of processes: ");
    scanf("%d", &n);
    int process[n];

    printf("Enter sizes of processes:\n");
    for(int i = 0; i < n; i++){
        scanf("%d", &process[i]);
    }

    int allocation[n];
    for(int i = 0; i < n; i++) allocation[i] = -1;
    for(int i = 0; i < n; i++) {
        int best = -1;
        for(int j = 0; j < m; j++) {
            if(blocks[j] >= process[i]) {
                if(best == -1 || blocks[j] < blocks[best])
                    best = j;
            }
        }
        if(best != -1) {
            allocation[i] = best;
            blocks[best] -= process[i];
        }
    }
    printf("\nBest Fit Allocation:\n");
    for(int i = 0; i < n; i++) {
        if(allocation[i] != -1)
            printf("Process %d -> Block %d\n", i+1, allocation[i]+1);
        else
            printf("Process %d -> Not Allocated\n", i+1);
    }

    return 0;
}
```


BVRIT HYDERABAD College of Engineering for Women

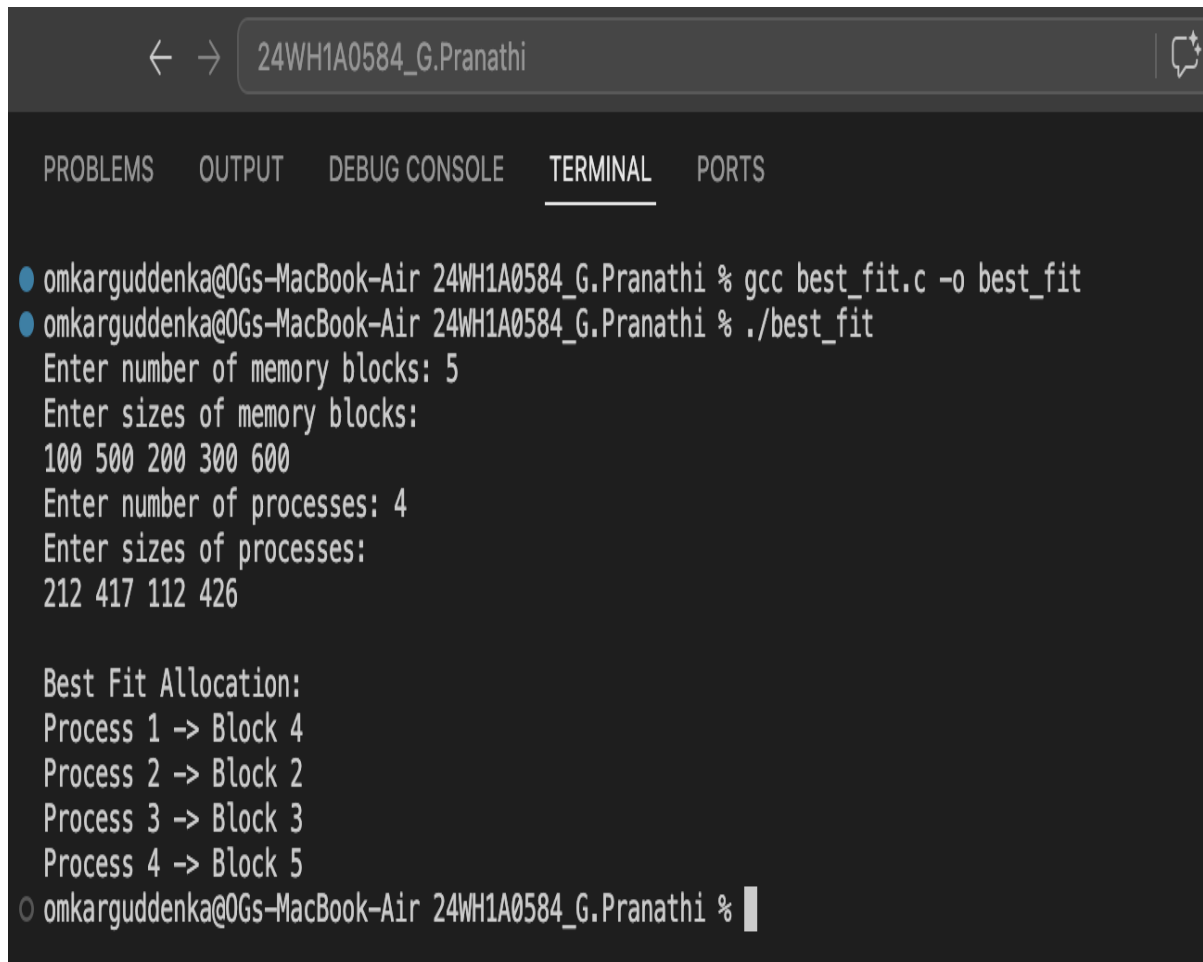
Title : CONTIGUOUS MEMORY ALLOCATION TECHNIQUES

Date : 10/04/26

ROLL NO: 24WH1A0584

Page No.: 38

OUTPUT:



```
← → 24WH1A0584_G.Pranathi
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc best_fit.c -o best_fit
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./best_fit
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

Best Fit Allocation:
Process 1 -> Block 4
Process 2 -> Block 2
Process 3 -> Block 3
Process 4 -> Block 5
○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

BVRIT HYDERABAD College of Engineering for Women

Title : CONTIGUOUS MEMORY ALLOCATION TECHNIQUES

Date : 10/04/26

ROLL NO: 24WH1A0584

Page No.: 39

c) Worst-fit:

Program:

```
#include <stdio.h>
int main() {
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    int blocks[m];

    printf("Enter sizes of memory blocks:\n");
    for(int i = 0; i < m; i++)
        scanf("%d", &blocks[i]);

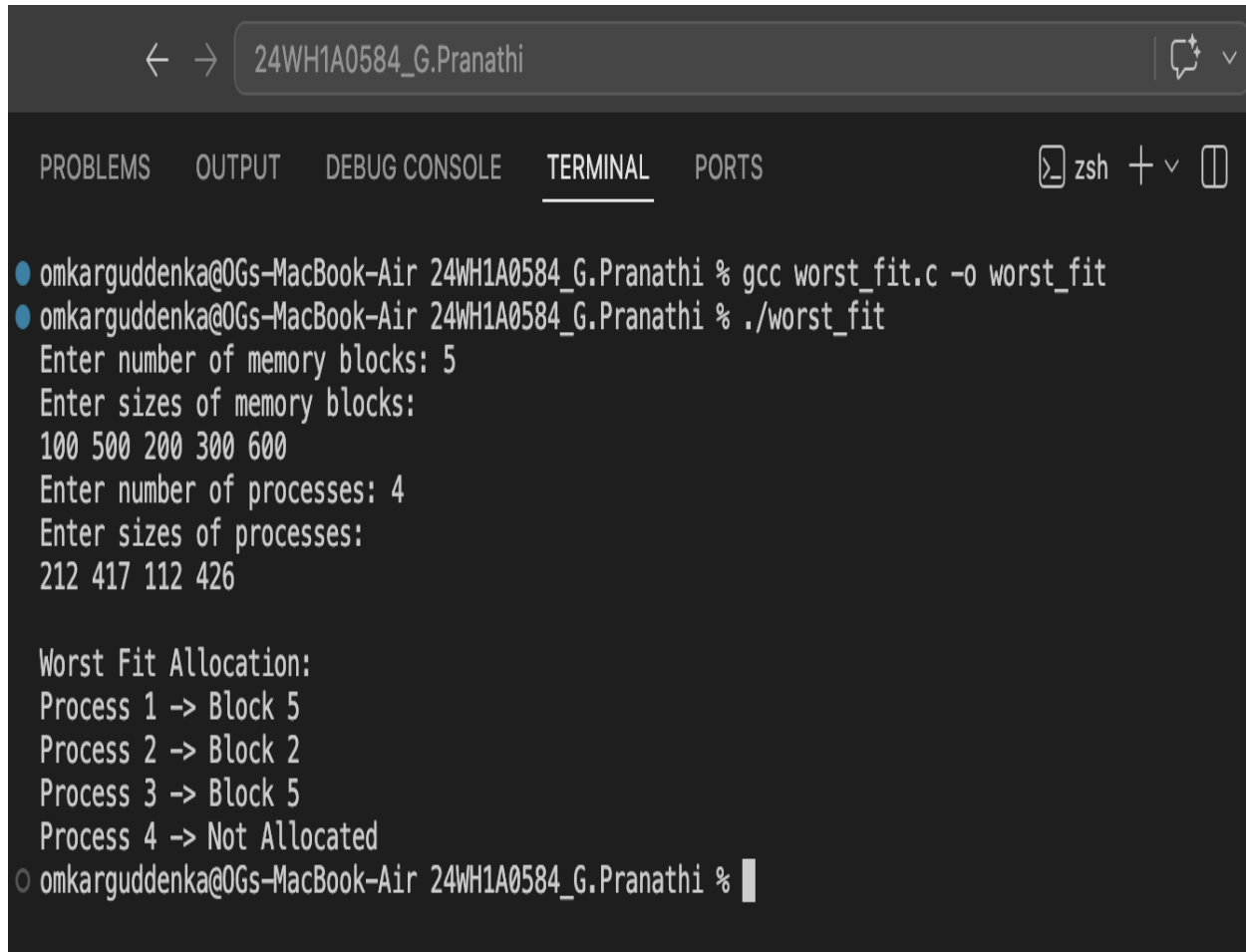
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int process[n];

    printf("Enter sizes of processes:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &process[i]);

    int allocation[n];
    for(int i = 0; i < n; i++) allocation[i] = -1;
    for(int i = 0; i < n; i++) {
        int worst = -1;
        for(int j = 0; j < m; j++) {
            if(blocks[j] >= process[i]) {
                if(worst == -1 || blocks[j] > blocks[worst])
                    worst = j;
            }
        }

        if(worst != -1) {
            allocation[i] = worst;
            blocks[worst] -= process[i];
        }
    }
    printf("\nWorst Fit Allocation:\n");
    for(int i = 0; i < n; i++) {
        if(allocation[i] != -1)
            printf("Process %d -> Block %d\n", i+1, allocation[i]+1);
        else
            printf("Process %d -> Not Allocated\n", i+1);
    }
    return 0;
}
```

OUTPUT:



```
24WH1A0584_G.Pranathi

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS zsh + v

● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc worst_fit.c -o worst_fit
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./worst_fit
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

Worst Fit Allocation:
Process 1 -> Block 5
Process 2 -> Block 2
Process 3 -> Block 5
Process 4 -> Not Allocated
○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

Aim 8. Write a program to stimulate Page Replacement Algorithms

a) FCFS

b) LRU

c) Optimal

a) FCFS:

Program:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j, k = 0, faults = 0, flag;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    scanf("%d", &f);

    for(i = 0; i < f; i++){
        frames[i] = -1;
    }

    printf("\n-----\n");
    printf("Page      F1      F2      F3      Status\n");
    printf("-----\n");

    for(i = 0; i < n; i++) {
        flag = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                flag = 1;
                break;
            }
        }
        if(flag == 0) {
            frames[k] = pages[i];
            k = (k + 1) % f;
            faults++;
        }

        printf("%-8d", pages[i]);
        for(j = 0; j < f; j++) {
            if(frames[j] != -1)
                printf("%-8d", frames[j]);
            else
                printf("%-8s", "-");
        }

        if(flag == 0)
            printf("PF");
        printf("\n");
    }
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : PAGE REPLACEMENT ALGORITHMS

Date : 7/04/26

ROLL NO: 24WH1A0584

Page No.: 42

```
printf("-----\n");
printf("Total Page Faults = %d\n", faults);
printf("-----\n");

return 0;
}
```

OUTPUT:

```
← → 24WH1A0584_G.Pranathi
PROBLEMS OUTPUT TERMINAL ... zsh + v [ ] [ ] ... [ ] ×
```

```
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc fcfs_pageReplacement.c
-o fcfs_pageReplacement
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./fcfs_pageReplacement
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3
```

Page	F1	F2	F3	Status
7	7	-	-	PF
0	7	0	-	PF
1	7	0	1	PF
2	2	0	1	PF
0	2	0	1	
3	2	3	1	PF
0	2	3	0	PF
4	4	3	0	PF
2	4	2	0	PF
3	4	2	3	PF
0	0	2	3	PF
3	0	2	3	
2	0	2	3	
1	0	1	3	PF
2	0	1	2	PF
0	0	1	2	
1	0	1	2	
7	7	1	2	PF
0	7	0	2	PF
1	7	0	1	PF

```
-----
Total Page Faults = 15
-----
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

BVRIT HYDERABAD College of Engineering for Women

Title : PAGE REPLACEMENT ALGORITHMS

Date : 7/04/26

ROLL NO: 24WH1A0584

Page No.: 43

b) LRU:

Program:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], time[10];
    int n, f, i, j, pos, faults = 0, counter = 0, flag;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\n-----\n");
    printf("Page      F1      F2      F3      Status\n");
    printf("-----\n");

    for(i = 0; i < n; i++) {
        flag = 0;

        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                flag = 1;
                break;
            }
        }

        if(flag == 0) {
            int min = time[0];
            pos = 0;

            for(j = 1; j < f; j++) {
                if(time[j] < min) {
                    min = time[j];
                    pos = j;
                }
            }
            counter++;
            frames[pos] = pages[i];
            time[pos] = counter;
            faults++;
        }
    }
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : PAGE REPLACEMENT ALGORITHMS

Date : 7/04/26

ROLL NO: 24WH1A0584

Page No.: 44

```
    printf("%-8d", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%-8d", frames[j]);
        else
            printf("%-8s", "-");
    }
    if(flag == 0)
        printf("PF");
    printf("\n");
}
printf("-----\n");
printf("Total Page Faults = %d\n", faults);
printf("-----\n");
return 0;
}
```

OUTPUT:

```
← → 24WH1A0584_G.Pranathi

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc lru.c -o lru
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./lru
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

-----
Page      F1      F2      F3      Status
-----
7         7         -         -         PF
0         7         0         -         PF
1         7         0         1         PF
2         2         0         1         PF
0         2         0         1
3         2         0         3         PF
0         2         0         3
4         4         0         3         PF
2         4         0         2         PF
3         4         3         2         PF
0         0         3         2         PF
3         0         3         2
2         0         3         2
1         1         3         2         PF
2         1         3         2
0         1         0         2         PF
1         1         0         2
7         1         0         7         PF
0         1         0         7
1         1         0         7
-----
Total Page Faults = 12
-----
○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

BVRIT HYDERABAD College of Engineering for Women

Title : PAGE REPLACEMENT ALGORITHMS

Date : 7/04/26

ROLL NO: 24WH1A0584

Page No.: 45

c) Optimal:

Program:

```
#include <stdio.h>
int main() {
    int pages[50], frames[10];
    int n, f, i, j, k, pos, faults = 0, flag;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for(i = 0; i < n; i++){
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &f);
    for(i = 0; i < f; i++){
        frames[i] = -1;
    }
    printf("\n-----\n");
    printf("Page      F1      F2      F3      Status\n");
    printf("-----\n");
    for(i = 0; i < n; i++) {
        flag = 0;
        for(j = 0; j < f; j++) {
            if(frames[j] == pages[i]) {
                flag = 1;
                break;
            }
        }
        if(flag == 0) {
            int farthest = -1;
            pos = -1;
            for(j = 0; j < f; j++) {
                int found = 0;
                for(k = i + 1; k < n; k++) {
                    if(frames[j] == pages[k]) {
                        if(k > farthest) {
                            farthest = k;
                            pos = j;
                        }
                    }
                }
                found = 1;
                break;
            }
            if(!found) {
                pos = j;
                break;
            }
        }
        if(pos == -1)
            pos = 0;

        frames[pos] = pages[i];
        faults++;
    }
}
```


BVRIT HYDERABAD College of Engineering for Women

Title : PAGE REPLACEMENT ALGORITHMS

Date : 7/04/26

ROLL NO: 24WH1A0584

Page No.: 46

```
        printf("%-8d", pages[i]);
    for(j = 0; j < f; j++) {
        if(frames[j] != -1)
            printf("%-8d", frames[j]);
        else
            printf("%-8s", "-");
    }

    if(flag == 0)
        printf("PF");
    printf("\n");
}
printf("-----\n");
printf("Total Page Faults = %d\n", faults);
printf("-----\n");
return 0;
}
```

OUTPUT:

```
← → 24WH1A0584_G.Pranathi

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc optimal.c -o optimal
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./optimal
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
Enter number of frames: 3

-----
Page      F1      F2      F3      Status
-----
7         7         -         -         PF
0         7         0         -         PF
1         7         0         1         PF
2         2         0         1         PF
0         2         0         1
3         2         0         3         PF
0         2         0         3
4         2         4         3         PF
2         2         4         3
3         2         4         3
0         2         0         3         PF
3         2         0         3
2         2         0         3
1         2         0         1         PF
2         2         0         1
0         2         0         1
1         2         0         1
7         7         0         1         PF
0         7         0         1
1         7         0         1

-----
Total Page Faults = 9
-----

○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

Aim 4. Write a program to implement Readers-Writers problem using Semaphores

Program:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
sem_t mutex, wrt;
int read_count = 0;
void *reader(void *arg) {
    int id = *(int *)arg;
    printf("READER %d is trying to enter...\n", id);
    sem_wait(&mutex);
    read_count++;
    if (read_count == 1)
        sem_wait(&wrt);
    printf("READER %d started reading.\n", id);
    printf("Number of readers = %d\n\n", read_count);
    sem_post(&mutex);
    sleep(2 + id);
    sem_wait(&mutex);
    read_count--;
    printf("READER %d finished reading.\n", id);
    printf("Number of readers = %d\n\n", read_count);
    if (read_count == 0)
        sem_post(&wrt);
    sem_post(&mutex);
    return NULL;
}
void *writer(void *arg) {
    int id = *(int *)arg;
    sleep(2);
    printf("WRITER %d is trying to enter...\n", id);
    sem_wait(&wrt);
    printf("WRITER %d started writing...\n", id);
    sleep(2);
    printf("WRITER %d finished writing.\n\n", id);
    sem_post(&wrt);
    return NULL;
}
int main() {
    int r, w;
    printf("Enter number of readers: ");
    scanf("%d", &r);
    printf("Enter number of writers: ");
    scanf("%d", &w);
    pthread_t rtid[r], wtid[w];
    int rid[r], wid[w];
    sem_init(&mutex, 0, 1);
    sem_init(&wrt, 0, 1);
    for (int i = 0; i < r; i++)
        rid[i] = i + 1;
    for (int i = 0; i < w; i++)
        wid[i] = i + 1;
```

BVRIT HYDERABAD College of Engineering for Women

Title : READERS-WRITERS PROBLEM(BEYOND THE SYLLABUS)

Date : 20/02/26

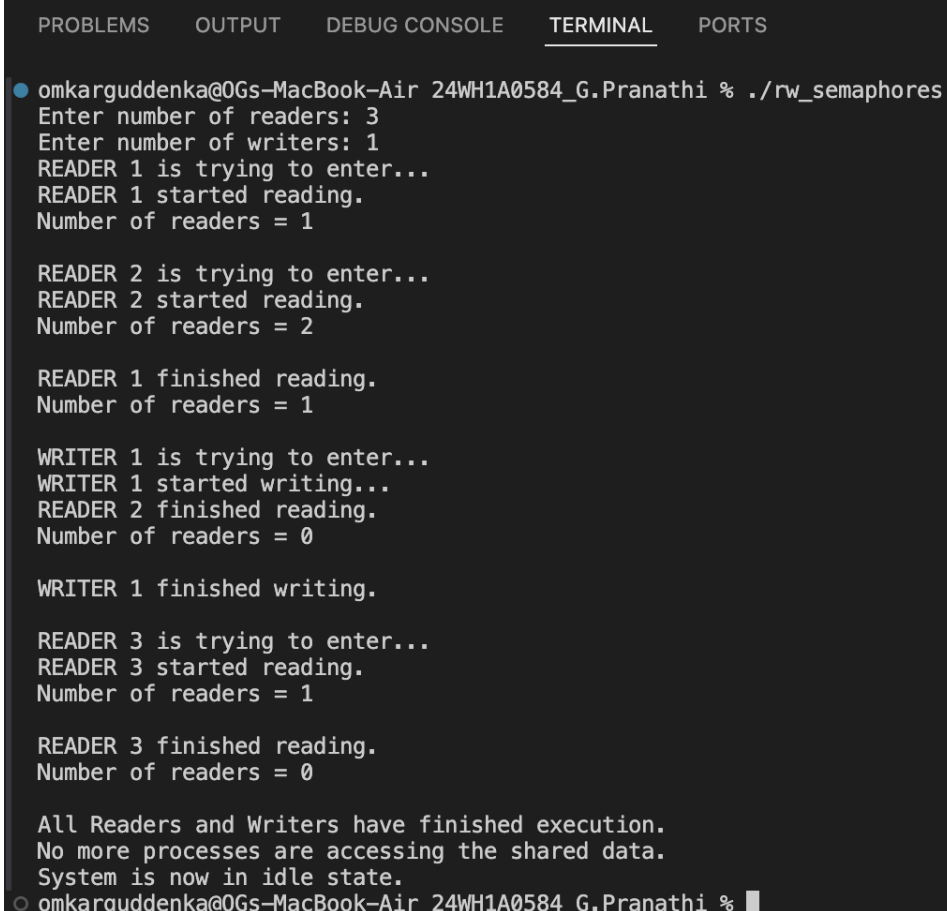
ROLL NO: 24WH1A0584

Page No.: 54

```
for (int i = 0; i < r - 1; i++) {
    pthread_create(&rtid[i], NULL, reader, &rid[i]);
    sleep(1);
}
for (int i = 0; i < w; i++) {
    pthread_create(&wtid[i], NULL, writer, &wid[i]);
}
sleep(8);
if (r > 0)
    pthread_create(&rtid[r - 1], NULL, reader, &rid[r - 1]);
for (int i = 0; i < r; i++) {
    pthread_join(rtid[i], NULL);
}
for (int i = 0; i < w; i++) {
    pthread_join(wtid[i], NULL);
}
printf("All Readers and Writers have finished execution.\n");
printf("No more processes are accessing the shared data.\n");
printf("System is now in idle state.\n");

sem_destroy(&mutex);
sem_destroy(&wrt);
return 0;
}
```

OUTPUT:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./rw_semaphores
Enter number of readers: 3
Enter number of writers: 1
READER 1 is trying to enter...
READER 1 started reading.
Number of readers = 1

READER 2 is trying to enter...
READER 2 started reading.
Number of readers = 2

READER 1 finished reading.
Number of readers = 1

WRITER 1 is trying to enter...
WRITER 1 started writing...
READER 2 finished reading.
Number of readers = 0

WRITER 1 finished writing.

READER 3 is trying to enter...
READER 3 started reading.
Number of readers = 1

READER 3 finished reading.
Number of readers = 0

All Readers and Writers have finished execution.
No more processes are accessing the shared data.
System is now in idle state.
○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

Aim 5. Write a program to implement Dining Philosophers problem using Semaphores

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <unistd.h>
sem_t *forks;
int N;
void eat(int i) {
    printf("\nPhilosopher %d is HUNGRY\n", i);
    sem_wait(&forks[i]);
    printf("Philosopher %d picked up LEFT fork %d\n", i, i);
    sem_wait(&forks[(i + 1) % N]);
    printf("Philosopher %d picked up RIGHT fork %d\n", i, (i + 1) % N);
    printf("Philosopher %d is EATING\n", i);
    sleep(2);
    sem_post(&forks[i]);
    printf("Philosopher %d put down LEFT fork %d\n", i, i);
    sem_post(&forks[(i + 1) % N]);
    printf("Philosopher %d put down RIGHT fork %d\n", i, (i + 1) % N);
    printf("Philosopher %d finished EATING and is THINKING\n", i);
}
int main() {
    int choice;
    printf("Enter number of philosophers: ");
    scanf("%d", &N);
    forks = (sem_t *)malloc(N * sizeof(sem_t));
    for (int i = 0; i < N; i++) {
        sem_init(&forks[i], 0, 1);
    }
    for (int i = 0; i < N; i++) {
        printf("Philosopher %d is THINKING\n", i);
    }
    printf("\nEnter philosopher number (0 to %d): ", N - 1);
    scanf("%d", &choice);
    if (choice >= 0 && choice < N) {
        eat(choice);
    } else {
        printf("Invalid philosopher number!\n");
    }

    free(forks);
    return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : DINING PHILOSOPHERS(BEYOND THE SYLLABUS)

Date: 13/03/26

ROLL NO: 24WH1A0584

Page No.: 56

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc dp_semaphores.c -o dp_semaphore
s -pthread
● omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./dp_semaphores
Enter number of philosophers: 5
Philosopher 0 is THINKING
Philosopher 1 is THINKING
Philosopher 2 is THINKING
Philosopher 3 is THINKING
Philosopher 4 is THINKING

Enter philosopher number (0 to 4): 3

Philosopher 3 is HUNGRY
Philosopher 3 picked up LEFT fork 3
Philosopher 3 picked up RIGHT fork 4
Philosopher 3 is EATING
Philosopher 3 put down LEFT fork 3
Philosopher 3 put down RIGHT fork 4
Philosopher 3 finished EATING and is THINKING
○ omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```

Aim 6. Write a program to implement Dining Philosophers problem using Monitors

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#define THINKING 0
#define HUNGRY 1
#define EATING 2
int *state;
int N;
pthread_mutex_t mutex;
pthread_cond_t *cond;
int LEFT(int i) { return (i + N - 1) % N; }
int RIGHT(int i) { return (i + 1) % N; }
void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {
        state[i] = EATING;
        printf("Philosopher %d picked up forks (LEFT %d and RIGHT %d)\n",
            i, i, RIGHT(i));
        printf("Philosopher %d is EATING\n\n", i);
        pthread_cond_signal(&cond[i]);
    }
}
void take_forks(int i) {
    pthread_mutex_lock(&mutex);
    state[i] = HUNGRY;
    printf("Philosopher %d is HUNGRY\n", i);
    test(i);
    if (state[i] != EATING) {
        printf("Philosopher %d is WAITING (neighbor is eating)\n\n", i);
        pthread_cond_wait(&cond[i], &mutex);
    }
    pthread_mutex_unlock(&mutex);
}
void put_forks(int i) {
    pthread_mutex_lock(&mutex);
    state[i] = THINKING;
    printf("Philosopher %d finished EATING\n", i);
    printf("Philosopher %d put down forks and started THINKING\n\n", i);
    test(LEFT(i));
    test(RIGHT(i));
    pthread_mutex_unlock(&mutex);
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : DINING PHILOSOPHERS(BEYOND THE SYLLABUS)

Date: 20/03/26

ROLL NO: 24WH1A0584

Page No.: 58

```
void *philosopher(void *arg) {
    int i = *(int *)arg;
    printf("Philosopher %d is THINKING\n", i);
    sleep(1);
    take_forks(i);
    sleep(2);
    put_forks(i);
    return NULL;
}

int main() {
    printf("Enter number of philosophers: ");
    scanf("%d", &N);
    pthread_t *tid = malloc(N * sizeof(pthread_t));
    int *phil = malloc(N * sizeof(int));
    state = malloc(N * sizeof(int));
    cond = malloc(N * sizeof(pthread_cond_t));
    pthread_mutex_init(&mutex, NULL);
    for (int i = 0; i < N; i++) {
        state[i] = THINKING;
        pthread_cond_init(&cond[i], NULL);
    }
    for (int i = 0; i < N; i++) {
        phil[i] = i;
        pthread_create(&tid[i], NULL, philosopher, &phil[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(tid[i], NULL);
    }
    printf("\nAll philosophers completed eating.\n");
    return 0;
}
```

BVRIT HYDERABAD College of Engineering for Women

Title : DINING PHILOSOPHERS(BEYOND THE SYLLABUS)

Date: 20/03/26

ROLL NO: 24WH1A0584

Page No.: 59

OUTPUT:

```
← → 24WH1A0584_G.Pranathi |
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % gcc dp_monitors.c -o dp_monitors -p
  thread
• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi % ./dp_monitors
Enter number of philosophers: 5
Philosopher 0 is THINKING
Philosopher 1 is THINKING
Philosopher 2 is THINKING
Philosopher 3 is THINKING
Philosopher 4 is THINKING
Philosopher 2 is HUNGRY
Philosopher 2 picked up forks (LEFT 2 and RIGHT 3)
Philosopher 2 is EATING

Philosopher 1 is HUNGRY
Philosopher 1 is WAITING (neighbor is eating)

Philosopher 3 is HUNGRY
Philosopher 3 is WAITING (neighbor is eating)

Philosopher 4 is HUNGRY
Philosopher 4 picked up forks (LEFT 4 and RIGHT 0)
Philosopher 4 is EATING

Philosopher 0 is HUNGRY
Philosopher 0 is WAITING (neighbor is eating)

Philosopher 2 finished EATING
Philosopher 2 put down forks and started THINKING

Philosopher 1 picked up forks (LEFT 1 and RIGHT 2)
Philosopher 1 is EATING

Philosopher 4 finished EATING
Philosopher 4 put down forks and started THINKING

Philosopher 3 picked up forks (LEFT 3 and RIGHT 4)
Philosopher 3 is EATING

Philosopher 3 finished EATING
Philosopher 3 put down forks and started THINKING

Philosopher 1 finished EATING
Philosopher 1 put down forks and started THINKING

Philosopher 0 picked up forks (LEFT 0 and RIGHT 1)
Philosopher 0 is EATING

Philosopher 0 finished EATING
Philosopher 0 put down forks and started THINKING

All philosophers completed eating.
• omkarguddenka@OGs-MacBook-Air 24WH1A0584_G.Pranathi %
```